分帧优化统一解决方案

一、简介

目前项目内有些地方使用了分帧进行优化比较大的junk，但都是自行在各自独立的tick函数中实现，没有一个统一的解决方案，这里我提供了一个通用的解决方案，后续需要分帧优化时可以直接用。

二、核心用法

2.1 API

C/C++

```cpp
/**
 * 分帧Task蓝图库函数
 */
UCLASS(meta=(ScriptName="FrameTaskMgr", ToolTip="Create multi frame tasks for optimizing cpu usage"))
class GAME_API UFrameTaskLib : public UBlueprintFunctionLibrary
{
    GENERATED_BODY()

public:
    /**
     * 向名为'QueueName'的队列中增加分帧任务（BP）
     * @param QueueName 分帧队列的名字
     * @param TaskEvent 新增分帧任务的事件回调
     * @param SkipFrames 新增分帧任务间隔的帧数(相比队列中前一个分帧任务)
     */
    UFUNCTION(BlueprintCallable, meta=(ToolTip="Add frame task: 'SkipFrames' should be no less than 1!"))
    static void AddFrameTask(const FName QueueName, UPARAM(ref) const FFrameTaskUFuncEvent& TaskEvent, int32 SkipFrames = 1);

    /**
     * 向名为'QueueName'的队列中增加分帧任务（C++）
     * @param InQueueName 分帧队列的名字
     * @param InLambda 新增分帧任务的Lambda回调
     * @param InSkipFrames 新增分帧任务间隔的帧数(相比队列中前一个分帧任务)
     * @return 新增分帧任务的对象
     */
    template<typename CallbackType>
    static FFrameTask& AddFrameTask(const FName& InQueueName, CallbackType&& InLambda, int32 InSkipFrames = 1);

    /**
     * RemoveFrameTaskQueue
     * 打断并清空名为'QueueName'的分帧队列（BP/C++）
     * @param QueueName 分帧队列的名字
     */
    UFUNCTION(BlueprintCallable)
```

```
    static void RemoveFrameQueue(const FName QueueName);
};
```

2.2 具体用法

- **C++**

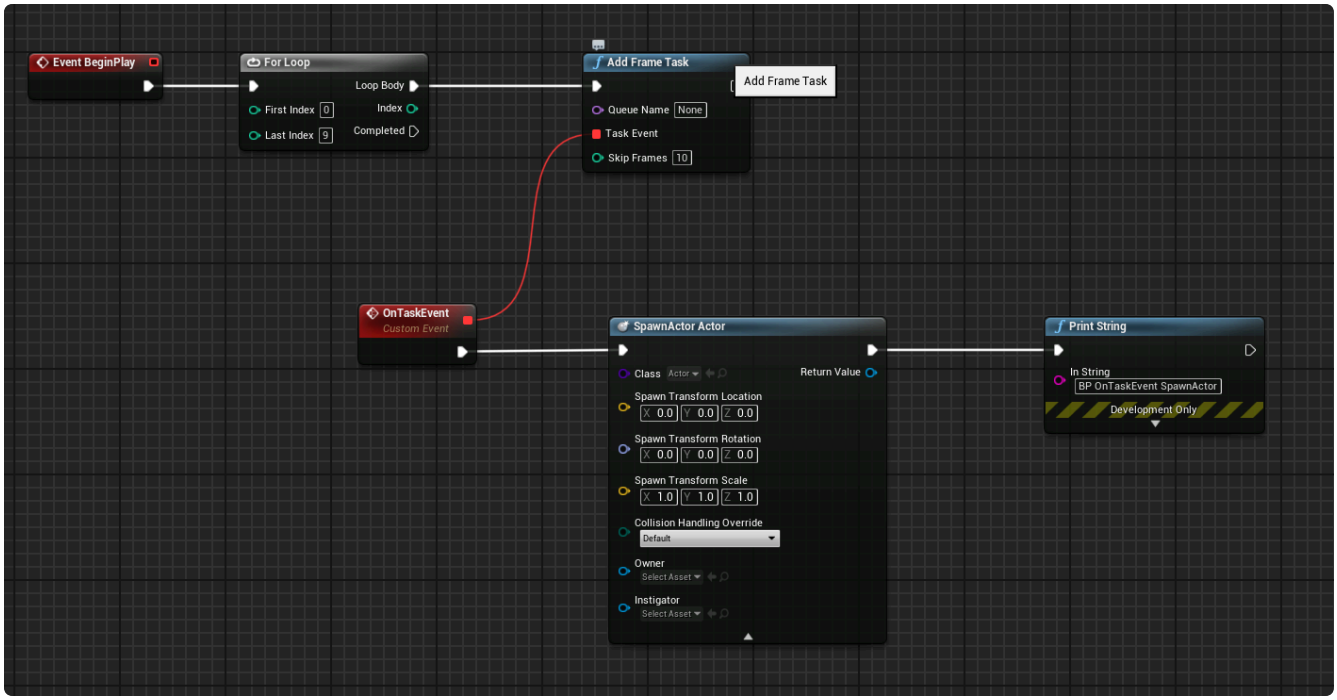
C/C++

```cpp
void ATestActor::BeginPlay()
{
    Super::BeginPlay();

    QUICK_SCOPE_CYCLE_COUNTER(STAT_ATESTACTOR_BEGINPLAY);

    if (bMultiFrame)
    {
        for (int32 i = 0; i < Loops; i += ExecuteCountOneFrame)
        {
            // C++层添加分帧任务
            UFrameTaskLib::AddFrameTask(TEXT("ATestActor::BeginPlay"), [i, this]()
            {
                for (int32 j = 0; j < ExecuteCountOneFrame; j++)
                {
                    GetWorld()->SpawnActor<AActor>();
                    UE_LOG(LogFrameTask, Log, TEXT("[Frame:%lld] Actor:%d"), this->FrameCount, i + j);
                }
            });
        }
    }
    else
    {
        for (int32 i = 0; i < Loops; i++)
        {
            GetWorld()->SpawnActor<AActor>();
        }
    }
}
```

- **蓝图**



> Lua：
> UFrameTaskLib是蓝图类，按照正常unlua的使用规范调用UFrameTaskLib::AddFrameTask即可